

UNIVERSITATEA TEHNICĂ A MOLDOVEI

Facultatea Calculatoare, Informatică și Microelectronică

Departamentul Ingineria Software și Automatică

LUCRARE DE LABORATOR NR. 3

The Command Line: Shell, Pipes and Text Filters

Disciplina: Sisteme de Operare

A efectuat:
student gr. FAF-243
Daniel Nicorici

Au verificat:
asist. univ. Patricia Reițman
asist. univ. Marius Dumitrașcu

Chișinău — 2026

Cuprins

Purpose of the work **2**

1 Part 1. Move fast, find help **2**

2 Part 2. What the shell does to your line **4**

3 Part 3. Redirection, chaining and jobs **5**

4 Part 4. Answer questions with one-liners **7**

5 Part 5. Your first script **9**

Conclusions **11**

Purpose of the work

The purpose of this lab is to work fluently in bash: to move fast with completion and history, to predict what the shell does to a command line before the program runs, to send output and errors exactly where I want them, and to answer real questions about a data set with one line of small filters joined by pipes, finally turning the best one-liners into a script.

The work was done on my own laptop running Arch Linux, in GNU bash 5.3.15. The data set for Part 4 was generated with the loop from the assignment:

```
$ mkdir -p ~/os-lab3 && cd ~/os-lab3
$ for i in $(seq 1 500); do
    echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) $([ $((i % 5)) -eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
  done > access.log
$ head -3 access.log ; wc -l access.log
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

The check prints `500 access.log`, so the file is correct.

1 Part 1. Move fast, find help

Pressing Tab asks bash to complete the current word. `/usr/sh` has only one match, so the first Tab completed it to `/usr/share/`; the next Tab only rang the bell because many entries start with `/usr/share/`, and the one after that offered to list all 229 of them. `cd /et` plus Tab completed to `cd /etc/` at once.

```
$ ls /usr/sh[Tab]
$ ls /usr/share/[Tab]
$ ls /usr/share/[Tab]
Display all 229 possibilities? (y or n) n
$ cd /et[Tab]
$ cd /etc/
$ pwd
/etc
```

Figure 1.1 – Tab completion

```
$ type cd ; type ls ; type echo ; which ls
cd is a shell builtin
ls is aliased to `ls --color=auto'
echo is a shell builtin
/usr/bin/ls
```

Figure 1.2 – Output of type and which

```
$ echo $PATH
/home/Nico/.local/bin:/home/Nico/.local/bin:/home/Nico/.bun/bin:/usr/local/sbin:/usr/local/bin:/usr/bin:/opt/cuda/bin:/usr/lib/jvm/default/bin:/usr/bin/site_perl:/usr/bin/vendor_perl:/usr/bin/core_perl
```

```
$ echo $PATH | tr ':' '\n'
/home/Nico/.local/bin
/home/Nico/.local/bin
/home/Nico/.bun/bin
/usr/local/sbin
/usr/local/bin
/usr/bin
/opt/cuda/bin
/usr/lib/jvm/default/bin
/usr/bin/site_perl
/usr/bin/vendor_perl
/usr/bin/core_perl
$ echo $PATH | tr ':' '\n' | wc -l
11
```

Figure 1.3 – The directories in PATH

```
$ nosuchprog ; echo "exit code: $?"
bash: nosuchprog: command not found
exit code: 127
```

Figure 1.4 – Exit code of a missing command

After running three commands, Ctrl+R started a reverse search through the history: the first letter `e` matched `date`, `ec` already matched `echo first command`, and Enter ran that command again.

```
$ echo first command
first command
$ date
Mon Oct  5 17:32:25 EEST 2026
$ whoami
Nico
(reverse-i-search)`ech': echo first command
first command
```

Figure 1.5 – Reverse history search with Ctrl+R

```
$ ls --help | head -5
Usage: ls [OPTION]... [FILE]...
List information about the FILEs (the current directory by default).
Sort entries alphabetically if none of -cftuvSUX nor --sort is specified.

Mandatory arguments to long options are mandatory for short options too.
$ man -k "list directory"
dir (1)              - list directory contents
ls (1)               - list directory contents
vdir (1)             - list directory contents
$ help cd | head -2
cd: cd [-L|[-P [-e]]] [-@] [dir]
    Change the shell working directory.
$ man ls | grep -n '^ *-t,\? '
161:     -t     sort by time, newest first; see --time
```

Figure 1.6 – Finding help without a browser

Observations:

1. **Built-ins.** `cd` and `echo` are shell built-ins; `ls` is an external program (`/usr/bin/ls`), which my `~/.bashrc` wraps in the alias `ls --color=auto`. `cd` *must* be a built-in because the current directory belongs to each process: an external `cd` would run as a child process, change the child's directory and exit, leaving the shell exactly where it was.
2. **PATH.** My PATH has 11 entries (Figure 1.3), but only 10 different directories: `~/.local/bin` is added twice, once by `~/.zshrc` and once by `~/.bashrc`. `ls` is found in `/usr/bin`.
3. **Missing command.** A command that is not found returns exit code 127.
4. **Sort by time.** `ls -t` sorts by modification time, newest first (line 161 of `man ls`); `ls -lt` shows it in long format.

2 Part 2. What the shell does to your line

```
$ mkdir -p ~/os-lab3/play && cd ~/os-lab3/play
$ touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
$ ls
 a.c  'alfa beta.txt'   b.c   file1.txt   file2.txt   notes.txt
```

Figure 2.1 – Files created in the play directory

```
$ echo *.c
a.c b.c
$ echo file?.txt
file1.txt file2.txt
$ echo [a-b]*
a.c alfa beta.txt b.c
$ echo *.xyz
*.xyz
```

Figure 2.2 – Globbing results

```
$ ls alfa beta.txt
ls: cannot access 'alfa': No such file or directory
ls: cannot access 'beta.txt': No such file or directory
$ ls "alfa beta.txt"
'alfa beta.txt'
$ ls alfa\ beta.txt
'alfa beta.txt'
```

Figure 2.3 – The ls error on a name with a space, and the fix

```
$ echo '$HOME' "$HOME" \$HOME
$HOME /home/Nico $HOME
```

Figure 2.4 – Single quotes, double quotes and backslash

```
$ echo {1..5} ; echo $((7 * 6))
1 2 3 4 5
42
```

```
$ echo "today is $(date +%A), I am $(whoami)"
today is Monday, I am Nico
$ touch log{1..3}.txt && ls log*
log1.txt  log2.txt  log3.txt
```

Figure 2.5 – Brace expansion, arithmetic and command substitution

Table 2.1 – Predictions compared with the real output

Command	My prediction	Real output	Explanation
`echo *.c`	`a.c b.c`	`a.c b.c`	bash replaces the pattern with matching names
`echo file?.txt`	`file1.txt file2.txt`	`file1.txt file2.txt`	`?` matches exactly one character
`echo [a-b]*`	`a.c alfa beta.txt b.c`	`a.c alfa beta.txt b.c`	names starting with a or b, sorted
`echo *.xyz`	`*.xyz`	`*.xyz`	no match, so bash leaves the pattern unchanged
`ls alfa beta.txt`	two “cannot access” errors	two “cannot access” errors	the space splits the name into two arguments
`ls "alfa beta.txt"`	`'alfa beta.txt'`	`'alfa beta.txt'`	quotes keep it one argument
`echo '$HOME' "$HOME" \$HOME`	`$HOME /home/Nico $HOME`	`$HOME /home/Nico $HOME`	only double quotes allow expansion
`echo {1..5} ; echo $((7 * 6))`	`1 2 3 4 5, 42`	`1 2 3 4 5, 42`	brace expansion, then arithmetic
`echo "today is $(date +%A), I am $(whoami)"`	`today is Monday, I am Nico`	`today is Monday, I am Nico`	`$(...)` is replaced by the command’s output
`touch log{1..3}.txt && ls log*`	`log1.txt log2.txt log3.txt`	`log1.txt log2.txt log3.txt`	braces create three names, the glob finds them

Observations:

1. **Predictions.** Table 2.1 compares each prediction with the real output. The one that is easy to get wrong is `echo *.xyz`: when a pattern matches nothing, bash does not remove it but passes it through unchanged, so `echo` prints `*.xyz` literally.
2. **Who produced the file names.** The shell. Before running `echo`, bash performs pathname expansion: it replaces `*.c` with the matching names in the current directory, so `echo` only receives `a.c` and `b.c` as ordinary arguments.
3. **Why `ls alfa beta.txt` failed.** Bash splits the line into words at spaces, so `ls` received two arguments, `alfa` and `beta.txt`, and neither file exists. Two fixes: quote the name (`"alfa beta.txt"` or `'alfa beta.txt'`), or escape the space with a backslash (`alfa\ beta.txt`). A glob such as `alfa*` also works, because a glob result is never split again.
4. **Single vs. double quotes.** Single quotes make everything inside literal, so `'$HOME'` stays `$HOME`. Double quotes keep the text as one word but still expand `$VAR`, `$(...)` and `$((...))`, so `"$HOME"` becomes `/home/Nico`. A backslash quotes only the next character.

`ls` shows `'alfa beta.txt'` in quotes because, when its output goes to a terminal, GNU `ls` quotes names that contain spaces or special characters, so that the name can be copied and pasted back into the shell as one argument.

3 Part 3. Redirection, chaining and jobs

```
$ cd ~/os-lab3
$ ls /etc /nope > out.txt 2> err.txt
$ wc -l out.txt ; cat err.txt
```

```
176 out.txt
ls: cannot access '/nope': No such file or directory
```

Figure 3.1 – stdout and stderr in separate files

```
$ ls /etc /nope > all.txt 2>&1 ; tail -2 all.txt
xdg
zsh
$ ls /etc /nope > all.txt 2>&1
$ grep -c nope all.txt
1
$ ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
$ grep -c nope all.txt
0
```

Figure 3.2 – Comparing the order of redirections

```
$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2
$ mkdir -p tmp && cd tmp && echo "I am in $(pwd)" ; cd ~/os-lab3
I am in /home/Nico/os-lab3/tmp
$ true && echo "&& ran: previous succeeded"
&& ran: previous succeeded
$ false && echo "this is skipped"
$ false || echo "|| ran: previous failed"
|| ran: previous failed
```

Figure 3.3 – The ; && || operators

```
$ sleep 100 &
[1] 130123
$ jobs ; kill %1 ; jobs
[1]+  Running                 sleep 100 &
[1]+  Running                 sleep 100 &
$ ls /etc /nope > all.txt 2>&1
[1]+  Terminated              sleep 100
```

Figure 3.4 – Running a background job and stopping it

```
$ echo first > notes.md ; echo second >> notes.md ; cat notes.md
first
second
$ wc -l < access.log
500
$ wc -l access.log
500 access.log
```

Figure 3.5 – Appending and input redirection

The second `jobs` in Figure 3.4 still shows `Running`: `kill` only sends the signal, and the process had not died yet when `jobs` ran a moment later. Bash reported `Terminated` just before the next prompt.

Observations:

1. **Which file got what.** `err.txt` received the error message, because `2>` redirects file descriptor 2 (standard error). `out.txt` received the listing from standard output: 176 lines, the `/etc:` header plus 175 entries.
2. **`> all.txt 2>&1` vs. `2>&1 > all.txt`.** Redirections are applied from left to right, and `2>&1` means "make stderr point where stdout points *now*". In the first form stdout is already the file, so both streams go into `all.txt` (`grep -c` finds the error once). In the second form stderr is copied from stdout while stdout is still the terminal, and only afterwards is stdout moved to the file, so the error appears on the screen and `all.txt` contains no error (count 0).
3. **`wc -l < access.log`.** With `<` the shell opens the file and hands it to `wc` as standard input, so `wc` never learns a file name and prints only the number. With `wc -l access.log`, `wc` opens the file itself and prints its name.
4. **Operators.** `;` runs the next command no matter what happened before. `&&` runs the next command only if the previous one succeeded (exit code 0). `||` runs the next command only if the previous one failed (non-zero exit code); here `ls` failed with code 2.

4 Part 4. Answer questions with one-liners

Every answer is one command line, saved in `oneliners.sh` with the question as a comment above it. Fields in `access.log` are separated by single spaces: 1 = date, 2 = time, 3 = user, 4 = result, 5 = page.

```
$ grep -c FAIL access.log
100
```

Figure 4.1 – Number of failed requests

```
$ cut -d' ' -f5 access.log | sort -u | nl
     1  /page0
     2  /page1
     3  /page2
     4  /page3
```

Figure 4.2 – Different pages requested

```
$ cut -d' ' -f5 access.log | sort | uniq -c
    125 /page0
    125 /page1
    125 /page2
    125 /page3
```

Figure 4.3 – Requests per page

```
$ grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -k1,1nr -k2,2 | awk 'NR == 1 { max
    ↪ = $1 } $1 == max'
     15 user3
     15 user5
```

Figure 4.4 – Users with the most failed requests

```
$ echo "user3: $(grep -cw user3 access.log) requests, $(grep -w user3 access.log | grep -c FAIL)
    ↪ failed"
user3: 72 requests, 15 failed
```

Figure 4.5 – Requests and failures of user3

```
$ grep FAIL access.log | tail -n 3 | cut -d' ' -f2,3
10:10 user0
10:15 user5
10:20 user3
```

Figure 4.6 – The last 3 failed requests, time and user

```
$ cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
     24 /usr/bin/nologin
      2 /usr/bin/bash
      1 /usr/bin/zsh
      1 /usr/bin/git-shell
      1 /bin/false
```

Figure 4.7 – Login shells in /etc/passwd

```
$ echo "ls: $(ls /etc | wc -l) lines, ls -l: $(ls -l /etc | wc -l) lines, first line of ls -l: $(
    ↪ ls -l /etc | head -n 1)"
ls: 175 lines, ls -l: 176 lines, first line of ls -l: total 1280
```

Figure 4.8 – ls and ls -l line counts

```
$ bash oneliners.sh
100
     1  /page0
     2  /page1
     3  /page2
     4  /page3
    125 /page0
    125 /page1
    125 /page2
    125 /page3
     15 user3
     15 user5
user3: 72 requests, 15 failed
10:10 user0
10:15 user5
10:20 user3
     24 /usr/bin/nologin
      2 /usr/bin/bash
      1 /usr/bin/zsh
      1 /usr/bin/git-shell
      1 /bin/false
ls: 175 lines, ls -l: 176 lines, first line of ls -l: total 1280
     1  2026-10-05 10:10 user3 FAIL /page2
     2  2026-10-05 10:20 user6 FAIL /page0
     3  2026-10-05 10:30 user2 FAIL /page2
     4  2026-10-05 10:40 user5 FAIL /page0
     5  2026-10-05 10:50 user1 FAIL /page2
```

```
[... lines 6-48 omitted ...]
    49  2026-10-05 10:10 user0 FAIL /page2
    50  2026-10-05 10:20 user3 FAIL /page0
```

Figure 4.9 – Full run of oneliners.sh (bonus lines 6–48 omitted)

```
$ grep -E '^[^ ]+ [0-9]{2}:[0-9]0 [^ ]+ FAIL ' access.log | nl | tail -n 2
    49  2026-10-05 10:10 user0 FAIL /page2
    50  2026-10-05 10:20 user3 FAIL /page0
```

Figure 4.10 – Bonus: failed requests at minutes ending in 0

Answers:

Q1. 100 requests failed.

Q2. 4 different pages: `/page0`, `/page1`, `/page2`, `/page3`. `sort -u` keeps one copy of each and `nl` numbers them, so the last number is the count.

Q3. Each page got 125 requests; $4 \times 125 = 500$, the number of lines in the log.

Q4. There is a tie: `user3` and `user5` have 15 failed requests each. The final `awk` keeps every line whose count equals the first (largest) one, so a tie is shown instead of silently picking one user.

Q5. `user3` made 72 requests, 15 of which failed. `-w` matches `user3` only as a whole word.

Q6. The last three failures are at 10:10 (`user0`), 10:15 (`user5`) and 10:20 (`user3`).

Q7. Five login shells appear: `/usr/bin/nologin` (24 accounts, system users that may not log in), `/usr/bin/bash` (2), `/usr/bin/zsh` (1, my account), `/usr/bin/git-shell` (1) and `/bin/false` (1).

Q8. My prediction was that `ls -l` adds one extra line, and it does: 175 vs. 176. When the output is a pipe, `ls` prints one name per line, while `ls -l` prints one line per entry *plus* the header `total 1280`, the disk space used by the listed files in 1 KiB blocks.

Bonus: 50 lines match. The pattern `[0-9]{2}:[0-9]0` accepts any time whose minute ends in 0, and `FAIL` must follow the user field. A minute ends in 0 when i is a multiple of 10, and a request fails when i is a multiple of 5, so both hold for the 50 multiples of 10 up to 500.

Self-check: the Q3 counts add up to 500, and the per-user failure counts behind Q4 (14 for `user0`, `user1`, `user2`, `user4`, `user6` and 15 for `user3`, `user5`) add up to $5 \times 14 + 2 \times 15 = 100$, the answer to Q1. The file runs with `bash oneliners.sh` inside `~/os-lab3`.

5 Part 5. Your first script

The TODO is completed with a loop over `user0`–`user6`: brace expansion produces the numbers 0 to 6, and for each user `grep -w` selects that user's lines and `grep -c FAIL` counts the failures.

```
#!/bin/bash
# usage: ./report.sh access.log
log=$1
if [ ! -f "$log" ]; then
        echo "no such file: $log" >&2
        exit 1
fi
```

```
total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")
echo "$total requests, $fails failed"
for n in {0..6}; do
        echo "user$n: $(grep -w "user$n" "$log" | grep -c FAIL) failures"
done
```

Figure 5.1 – Contents of report.sh

```
$ chmod +x report.sh
$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 5.2 – Running the script on access.log

```
$ ./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
$ ./report.sh nope.log 2> /dev/null ; echo "exit code: $?"
exit code: 1
```

Figure 5.3 – Error handling for a missing file

```
$ ./report.sh access.log ; echo "exit code: $?"
bash: ./report.sh: Permission denied
exit code: 126
$ report.sh access.log
bash: report.sh: command not found
```

Figure 5.4 – Running the script before chmod +x and without ./

Observations:

1. **The `#!/bin/bash` line.** It is the “shebang”: when the kernel is asked to execute the file, it sees `#!` and runs `/bin/bash` with the script as its argument, so the line chooses the interpreter. Before `chmod +x` the file has no execute permission, so the kernel refuses to run it: `Permission denied`, exit code 126 (Figure 5.4). It could still be run as `bash report.sh`.
2. **Why `./report.sh`.** A command name without a slash is searched only in the PATH directories, and the current directory is not in PATH (Figure 1.3), so `report.sh` alone gives `command not found`. `./` is an explicit path to the file in the current directory.
3. **`>&2` and `exit 1`.** The error goes to standard error so that it never mixes with the real output: if the report is piped or saved to a file, the error still reaches the screen, and it can be discarded separately with `2> /dev/null` (Figure 5.3). `exit 1` returns a non-zero exit code, which tells the caller (`&&`, `||`, another script) that the script failed even when the message is hidden.

Conclusions

This lab showed how much the shell does before a program even starts. Bash splits the line into words, expands globs, variables, braces, arithmetic and command substitutions, removes the quotes, and sets up redirections, so the program only receives a ready list of arguments and already-open file descriptors. Predicting each line before running it made these steps visible, especially the cases that surprise, such as an unmatched glob that stays literal or a space that splits one file name into two arguments.

Redirection is just rewiring file descriptors from left to right, which is why the order of `>` and `2>&1` matters, and exit codes are what `&&` and `||` decide on. With these pieces, questions about 500 lines of log data were answered by short pipelines of `grep`, `cut`, `sort`, `uniq` and `wc`, each tool doing one small job. The same pipelines became a reusable script once it had a shebang, execute permission, error messages on standard error and a meaningful exit code.